

Optimización de Rendimiento en MongoDB

Asignatura: Diseño y Optimización de Bases de Datos

Actividad: Guía de Actividad Unidad 5

Fecha: Semana del 10 de Junio al 14 de Junio

Integrantes del Equipo:
Juan Daniel Valderrama Pérez
Jorge Esteban Triviño Correa
Javier Andres Baron Fontanilla

Equipo E20

Docente: Miguel Alfonso Varela Fonseca

Universidad de la Sabana
Facultad de Ingeniería

2026

Tabla de Contenido

Resumen Ejecutivo	3
Implementación PostgreSQL	3
• Scripts DDL Ejecutados en Supabase y Diccionario de Datos Completo por Tabla	3
• Estrategia de Indexación y otras Técnicas con Justificación	5
Tabla 1. Matriz de Justificación de Consultas Seleccionadas para el Análisis	5
Tabla 2. Sentencias SQL Analizadas y Resultado Sin Optimización	6
Técnica 1: Reescritura de Subconsultas Complejas Aplicada a Q2	8
Técnica 2: Particionamiento Aplicado a orders	8
Tabla 3. Matriz de Decisión para la Implementación de Estrategia de Indexación	8
• Evidencias de mejoras después de la estrategia de indexación aplicada	9
Sentencia DDL Para la Creación de Índices	9
Tabla 4. Análisis de Impacto Cuantitativo	10
Implementación en MongoDB Atlas: Esquemas y Validación	10
• Validación de Esquemas	11
• Estrategia de Indexación y Técnicas de Optimización	11
Tabla 5. Estrategia de Indexación	11
• Análisis Detallado de Consultas - Casos de Estudio	11
Escenario A: Análisis de Reseñas Complejas - Índice Parcial	11
Escenario B: Enriquecimiento de Datos - Aggregation Pipeline	12
• Análisis de Impacto Cuantitativo	12
Tabla 6. Análisis de Impacto Cuantitativo en MongoDB	12
• Diseño de Teórico de Replica Set y Sharding	13
Configuración de Replica Sets	13
Diseño de Sharding - Particionamiento Horizontal a Gran Escala	14
Selección Analítica y Justificación de la Shard Key	14
Sincronización entre Sistemas	15
Flujo de Datos entre PostgreSQL y MongoDB	15
Estrategia de Consistencia	15
Justificación Arquitectónica	16
Lecciones Aprendidas	16
Obstáculos Encontrados y Soluciones Aplicadas	16
Limitaciones del Free Tier y Workarounds Implementados	16
Reflexión Técnica	17

Resumen Ejecutivo

El presente trabajo documenta la implementación y optimización de una arquitectura híbrida de persistencia para la plataforma Ecommify, utilizando PostgreSQL como sistema transaccional (OLTP) y MongoDB Atlas como plataforma analítica orientada a documentos (OLAP).

En PostgreSQL se implementaron estrategias avanzadas de optimización basadas en indexación especializada, particionamiento declarativo por rangos temporales y utilización de extensiones nativas como PostGIS y pg_trgm. Estas técnicas permitieron reducir significativamente los tiempos de ejecución de las consultas críticas del negocio, especialmente en escenarios de búsqueda de catálogo, trazabilidad de pedidos, análisis geográfico y procesamiento de pagos.

En MongoDB Atlas se desarrolló un modelo documental optimizado para el análisis de productos y reseñas, incorporando validación mediante JSON Schema, índices compuestos, índices parciales y aggregation pipelines optimizados. Adicionalmente, se diseñó una arquitectura teórica de escalabilidad basada en Replica Sets y Sharding mediante una estrategia Hashed Sharding sobre product_id_pg.

Los resultados obtenidos evidencian mejoras sustanciales en el rendimiento de ambas plataformas. En PostgreSQL se alcanzaron reducciones de tiempo superiores al 90% en varias consultas críticas, destacándose la optimización del catálogo de productos (97.18%), procesamiento de pagos (97.84%) y rastreo de pedidos (97.39%). En MongoDB, las estrategias de indexación permitieron reemplazar operaciones COLLSCAN por IXSCAN, reduciendo significativamente la cantidad de documentos inspeccionados y el tiempo de respuesta de las consultas analíticas.

La implementación demuestra que una arquitectura de persistencia políglota permite aprovechar las fortalezas de cada tecnología, utilizando PostgreSQL para garantizar consistencia transaccional y MongoDB para soportar cargas analíticas de alta flexibilidad y escalabilidad.

Implementación PostgreSQL

- Scripts DDL Ejecutados en Supabase y Diccionario de Datos Completo por Tabla

```
-- HABILITACIÓN DE EXTENSIONES SOLICITADAS
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- TABLA: product_category_name
CREATE TABLE product_category_name (
    product_category_name VARCHAR(100) PRIMARY KEY,
    product_category_name_english VARCHAR(100) NOT
    NULL
);
COMMENT ON TABLE product_category_name IS 'Tabla
maestra de traducción de categorías de productos del
portugués al inglés.';
COMMENT ON COLUMN
product_category_name.product_category_name IS
'Nombre original de la categoría en portugués. Llave
primaria.';
COMMENT ON COLUMN
product_category_name.product_category_name_english
IS 'Traducción oficial de la categoría al idioma
inglés.';

-- TABLA: products
CREATE TABLE products (
    product_id VARCHAR(50) PRIMARY KEY,
    sku VARCHAR(100) UNIQUE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE NOT NULL,
```

```
-- TABLA: orders - Particionamiento Mensual
Declarativo
CREATE TABLE orders (
    order_id VARCHAR(50) NOT NULL,
    customer_id VARCHAR(50) REFERENCES
customers(customer_id),
    order_status VARCHAR(30) NOT NULL,
    purchase_timestamp TIMESTAMPTZ NOT NULL,
    approved_at TIMESTAMPTZ,
    delivered_carrier_date TIMESTAMPTZ,
    delivered_customer_date TIMESTAMPTZ,
    estimated_delivery_date TIMESTAMPTZ,
    PRIMARY KEY (order_id, purchase_timestamp)
) PARTITION BY RANGE (purchase_timestamp);

COMMENT ON TABLE orders IS 'Tabla transaccional
centralizada de pedidos. Particionada mensualmente
de forma lógica.';
COMMENT ON COLUMN orders.order_id IS 'Identificador
único del pedido de compra. Parte de la llave
primaria compuesta.';
COMMENT ON COLUMN orders.customer_id IS 'Llave
foránea que referencia al comprador del pedido.';
COMMENT ON COLUMN orders.order_status IS 'Estado
actual del ciclo de vida de la orden (delivered,
shipped, invoiced, etc.).';
```

```
weight_g INT CHECK (weight_g >= 0),
dimensions_cm INT[] CHECK
(array_length(dimensions_cm, 1) = 3),
created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);
COMMENT ON TABLE products IS 'Tabla maestra mínima
transaccional que actúa como ancla relacional para
los ítems de compra.';
COMMENT ON COLUMN products.product_id IS
'Identificador único alfanumérico del producto
global. Llave primaria.';
COMMENT ON COLUMN products.sku IS 'Stock Keeping
Unit único generado para control transaccional e
inventario rígido.';
COMMENT ON COLUMN products.is_active IS 'Flag lógico
que indica si el producto está habilitado para
operaciones comerciales.';
COMMENT ON COLUMN products.weight_g IS 'Peso del
producto expresado en gramos. Utilizado para el
cálculo de costos de envío.';
COMMENT ON COLUMN products.dimensions_cm IS 'Array
tridimensional que almacena [alto, ancho,
profundidad] en centímetros para cálculos
logísticos.';
COMMENT ON COLUMN products.created_at IS 'Marca de
tiempo con zona horaria del registro del producto en
el ecosistema transaccional.';

-- TABLA: customers
CREATE TABLE customers (
customer_id VARCHAR(50) PRIMARY KEY,
customer_unique_id VARCHAR(50) NOT NULL,
customer_zip_code_prefix INT NOT NULL,
customer_city VARCHAR(100) NOT NULL,
customer_state VARCHAR(10) NOT NULL
);
COMMENT ON TABLE customers IS 'Almacena los datos de
los clientes asociados a una orden de compra
específica.';
COMMENT ON COLUMN customers.customer_id IS 'Llave
primaria del cliente por cada orden de compra (Olist
genera un ID por orden).';
COMMENT ON COLUMN customers.customer_unique_id IS
'Identificador único global e inmutable del cliente
en el sistema.';
COMMENT ON COLUMN customers.customer_zip_code_prefix
IS 'Primeros 5 dígitos del código postal del
domicilio del cliente.';
COMMENT ON COLUMN customers.customer_city IS 'Nombre
de la ciudad de residencia del cliente.';
COMMENT ON COLUMN customers.customer_state IS
'Código o abreviatura del estado/provincia del
cliente.';

-- TABLA: sellers
CREATE TABLE sellers (
seller_id VARCHAR(50) PRIMARY KEY,
seller_zip_code_prefix INT NOT NULL,
seller_city VARCHAR(100) NOT NULL,
seller_state VARCHAR(10) NOT NULL
);
COMMENT ON TABLE sellers IS 'Registro de vendedores
asociados que proveen productos en la plataforma de
Ecommify.';
COMMENT ON COLUMN sellers.seller_id IS
'Identificador único del vendedor. Llave primaria.';
COMMENT ON COLUMN sellers.seller_zip_code_prefix IS
'Código postal de ubicación de despacho del
vendedor.';
COMMENT ON COLUMN sellers.seller_city IS 'Ciudad
base de operaciones del vendedor.';
COMMENT ON COLUMN sellers.seller_state IS 'Estado o
región de operación del vendedor.';

-- TABLA: geolocation
CREATE TABLE geolocation (
geolocation_zip_code_prefix INT NOT NULL,
geolocation_lat DOUBLE PRECISION NOT NULL,
geolocation_lng DOUBLE PRECISION NOT NULL,
geolocation_city VARCHAR(100) NOT NULL,
geolocation_state VARCHAR(10) NOT NULL,
geom GEOMETRY(Point, 4326)
);
COMMENT ON TABLE geolocation IS 'Catálogo de
coordenadas geográficas de Brasil utilizado para
análisis geoespaciales y rutas de entrega.';
```

```
COMMENT ON COLUMN orders.purchase_timestamp IS
'Fecha y hora de confirmación del carrito. Clave
esencial de la partición.';
COMMENT ON COLUMN orders.approved_at IS 'Fecha de
aprobación del pago de la orden.';
COMMENT ON COLUMN orders.delivered_carrier_date IS
'Fecha en que el operador logístico recolectó el
paquete.';
COMMENT ON COLUMN orders.delivered_customer_date IS
'Fecha real de entrega final al domicilio del
cliente.';
COMMENT ON COLUMN orders.estimated_delivery_date IS
'Fecha objetivo de entrega prometida al cliente en
el checkout.';

-- Creación de particiones físicas
CREATE TABLE orders_2016 PARTITION OF orders FOR
VALUES FROM ('2016-01-01') TO ('2017-01-01');
CREATE TABLE orders_2017_p1 PARTITION OF orders FOR
VALUES FROM ('2017-01-01') TO ('2017-07-01');
CREATE TABLE orders_2017_p2 PARTITION OF orders FOR
VALUES FROM ('2017-07-01') TO ('2018-01-01');
CREATE TABLE orders_2018 PARTITION OF orders FOR
VALUES FROM ('2018-01-01') TO ('2019-01-01');

-- TABLA: order_items
CREATE TABLE order_items (
order_id VARCHAR(50) NOT NULL,
order_item_id INT NOT NULL,
product_id VARCHAR(50) REFERENCES
products(product_id),
seller_id VARCHAR(50) REFERENCES
sellers(seller_id),
shipping_limit_date TIMESTAMPTZ NOT NULL,
price NUMERIC(10,2) CHECK (price >= 0),
freight_value NUMERIC(10,2) CHECK (freight_value
>= 0),
purchase_timestamp TIMESTAMPTZ NOT NULL,
PRIMARY KEY (order_id, order_item_id,
purchase_timestamp),
FOREIGN KEY (order_id, purchase_timestamp)
REFERENCES orders(order_id, purchase_timestamp)
);
COMMENT ON TABLE order_items IS 'Desglose detallado
de los productos contenidos dentro de cada pedido de
Ecommify.';
COMMENT ON COLUMN order_items.order_id IS
'Identificador del pedido padre. Parte de la FK
compuesta hacia la tabla particionada.';
COMMENT ON COLUMN order_items.order_item_id IS
'Número secuencial que identifica el ítem dentro del
mismo pedido.';
COMMENT ON COLUMN order_items.product_id IS 'Llave
foránea que asocia el ítem al catálogo maestro de
productos.';
COMMENT ON COLUMN order_items.seller_id IS 'Llave
foránea que vincula al vendedor responsable del
producto.';
COMMENT ON COLUMN order_items.shipping_limit_date IS
'Fecha máxima establecida para que el vendedor
entregue el producto a la transportadora.';
COMMENT ON COLUMN order_items.price IS 'Costo
unitario transaccional del artículo adquirido.';
COMMENT ON COLUMN order_items.freight_value IS
'Costo de flete o transporte calculado
individualmente para el ítem.';
COMMENT ON COLUMN order_items.purchase_timestamp IS
'Copia de control temporal del pedido padre
requerida para la herencia del particionamiento.';

-- TABLA: payments
CREATE TABLE payments (
order_id VARCHAR(50) NOT NULL,
payment_sequential INT NOT NULL,
payment_type VARCHAR(30) NOT NULL,
payment_installments INT CHECK
(payment_installments >= 0),
payment_value NUMERIC(10,2) CHECK (payment_value
>= 0),
gateway_response JSONB,
purchase_timestamp TIMESTAMPTZ NOT NULL,
PRIMARY KEY (order_id, payment_sequential,
purchase_timestamp),
FOREIGN KEY (order_id, purchase_timestamp)
REFERENCES orders(order_id, purchase_timestamp)
);
```

COMMENT ON COLUMN geolocation.geolocation_zip_code_prefix IS 'Código de zona postal analizado.'; COMMENT ON COLUMN geolocation.geolocation_lat IS 'Latitud geográfica en formato decimal de alta precisión.'; COMMENT ON COLUMN geolocation.geolocation_lng IS 'Longitud geográfica en formato decimal de alta precisión.'; COMMENT ON COLUMN geolocation.geolocation_city IS 'Nombre estructurado de la ciudad.'; COMMENT ON COLUMN geolocation.geolocation_state IS 'Sigla identificadora del estado geográfico.'; COMMENT ON COLUMN geolocation.geom IS 'Objeto geométrico espacial indexable (Point) basado en el sistema de referencia estándar WGS84 (SRID 4326).'; -- TABLA: promotions CREATE TABLE promotions (promotion_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), product_id VARCHAR(50) REFERENCES products(product_id), discount_rate NUMERIC(4,2) CHECK (discount_rate > 0 AND discount_rate <= 1.00), duration_range TSTZRANGE NOT NULL); COMMENT ON TABLE promotions IS 'Gestión transaccional de descuentos temporales aplicables a productos en Ecommify.'; COMMENT ON COLUMN promotions.promotion_id IS 'Identificador único auto-generado mediante algoritmos criptográficos UUID.'; COMMENT ON COLUMN promotions.product_id IS 'Llave foránea que vincula la promoción a un producto existente.'; COMMENT ON COLUMN promotions.discount_rate IS 'Porcentaje de descuento representado en formato decimal (ej: 0.15 representa el 15%).'; COMMENT ON COLUMN promotions.duration_range IS 'Rango temporal nativo que define el límite exacto de inicio y fin de la promoción.';	COMMENT ON TABLE payments IS 'Registro de transacciones financieras y pasarelas de pago asociadas a las órdenes.'; COMMENT ON COLUMN payments.order_id IS 'Identificador del pedido asociado al cobro.'; COMMENT ON COLUMN payments.payment_sequential IS 'Secuencial en caso de que el cliente use múltiples métodos de pago para una misma orden.'; COMMENT ON COLUMN payments.payment_type IS 'Método seleccionado (credit_card, boleto, voucher, debit_card).'; COMMENT ON COLUMN payments.payment_installments IS 'Número de cuotas diferidas seleccionadas por el tarjetahabiente.'; COMMENT ON COLUMN payments.payment_value IS 'Monto total procesado y cobrado en la transacción monetaria.'; COMMENT ON COLUMN payments.gateway_response IS 'Metadato flexible JSONB que almacena la respuesta íntegra del API externo de pagos para auditorías.'; COMMENT ON COLUMN payments.purchase_timestamp IS 'Timestamp heredado obligatorio para mantener la integridad con la partición jerárquica de la orden.';
--	--

- Estrategia de Indexación y otras Técnicas con Justificación

Tabla 1. Matriz de Justificación de Consultas Seleccionadas para el Análisis

Consulta	Criterio de Selección	Justificación	Tablas Involucradas - #registros
Navegación de Catálogo por Categoría Q1	Alta Frecuencia	Podría representar más del 60% del tráfico de lectura. Requiere una respuesta rápida para evitar la deserción de usuarios en la app.	products (32,951) order_items (112,650)
Rastrear Estado de Pedido Q2	Alta Frecuencia	Consulta transaccional repetitiva por clientes ansiosos. Busca por customer_id en una tabla cercana a los 100k registros.	orders (99,441) customers (99,441) order_items (112,650) payments (103,886)
Reporte Geográfico de Entregas Q3	Alta Latencia	Operación de tipo OLAP. El volumen de registros en orders y customers provocará un Seq Scan destructivo para la CPU si no tiene índices compuestos o parciales.	orders (99,441) customers (99,441) geolocation (19,015)

Dashboard de Calidad de Producto Q4	Alta Latencia	Cruce de múltiples tablas masivas con funciones de agregación (AVG). El optimizador de Postgres tenderá a usar costosos algoritmos de Hash Join o Merge Join.	orders (99,441) order_items (112,650) products (32,951)
Procesamiento de Checkout / Pagos Q5	Impacto de Negocio	Flujo monetario principal de Ecommify. Escritura y lectura concurrente sobre tipos de pago e importes durante picos de venta.	orders (99,441) payments (103,886)

Tabla 2. Sentencias SQL Analizadas y Resultado Sin Optimización

Consulta	Sentencia SQL Analizada	Resultado Antes de Optimización
Q1	<pre>SELECT p.product_id, p.product_weight_g, t.product_category_name_english FROM products p JOIN product_category_name t ON p.product_category_name = t.product_category_name WHERE p.product_category_name = 'esporte_lazer' LIMIT 20;</pre>	Tiempo de ejecución: 1599.091 ms Uso de memoria/recursos: 1009kB para el HashAggregate , 160kB para el Hash operativo y 34kB en ordenamiento. Operación más costosa: Seq Scan sobre la tabla order_items y Seq Scan sobre products. El motor descarta 31,061 registros mediante un filtro. Escalará linealmente degradando el catálogo.
Q2	<pre>SELECT o.order_id, o.order_status, o.purchase_timestamp, oi.price, p.payment_type, p.payment_value FROM customers c JOIN orders o ON c.customer_id = o.customer_id JOIN order_items oi ON o.order_id = oi.order_id AND o.purchase_timestamp = oi.purchase_timestamp JOIN payments p ON o.order_id = p.order_id AND o.purchase_timestamp = p.purchase_timestamp WHERE c.customer_unique_id = '871766c5855e863f64db0585a72663bc' -- Cliente con múltiples compras históricas ORDER BY o.purchase_timestamp DESC;</pre>	Tiempo de ejecución: 477.887 ms Uso de memoria/recursos: Memory: 25kB para el método de ordenamiento rápido (<i>quicksort</i>). Operación más costosa: Parallel Seq Scan sobre la tabla customers removiendo 49,720 filas de forma ineficiente, combinado con un bloque Parallel Append. Al buscar al cliente por su ID único global sin proveer límites de tiempo directos en el filtro, PostgreSQL entra en pánico relacional y escanea secuencialmente todas las particiones físicas de la base de datos.
Q3	<pre>SELECT c.customer_city, COUNT(o.order_id) as total_ordenes FROM orders o JOIN customers c ON o.customer_id = c.customer_id JOIN geolocation g ON c.customer_zip_code_prefix = g.geolocation_zip_code_prefix WHERE o.order_status = 'delivered'</pre>	Tiempo de ejecución: 739.227 ms Uso de memoria/recursos: 2464kB en el Hash en paralelo y hasta 1283kB por nodo de trabajo en ordenamiento (<i>Worker 0</i>). Operación más costosa: Seq Scan sobre la tabla geolocation. El motor ejecuta la función analítica espacial ST_Contains de

	<pre>-- ST_MakeEnvelope crea un área rectangular [xmin, ymin, xmax, ymax] (Coordenadas de control) AND ST_Contains(ST_MakeEnvelope(-47.5, -24.0, -46.0, -23.0, 4326), g.geom) GROUP BY c.customer_city ORDER BY total_ordenes DESC LIMIT 20;</pre>	PostGIS directamente sobre el total de la tabla, evaluando geométricamente cada coordenada con el polígono mapeado sin asistencia de un índice indexable. Remueve 14,353 filas por filtro, lo que generará cuellos de botella masivos en la CPU al crecer los datos vectoriales.
Q4	<pre>SELECT p.product_id, p.sku, COUNT(oi.order_id) as items_vendidos, SUM(CASE WHEN o.delivered_customer_date > o.estimated_delivery_date THEN 1 ELSE 0 END) as entregas_con_retraso, AVG(oi.freight_value / (oi.price + 0.01)) as ratio_costo_envio FROM products p JOIN order_items oi ON p.product_id = oi.product_id JOIN orders o ON oi.order_id = o.order_id AND oi.purchase_timestamp = o.purchase_timestamp GROUP BY p.product_id, p.sku HAVING COUNT(oi.order_id) > 5 ORDER BY entregas_con_retraso DESC LIMIT 20;</pre>	Tiempo de ejecución: 1890.701 ms. <i>La consulta más lenta.</i> Uso de memoria/recursos: Sort Method external merge Disk: 13176kB. Operación más costosa: El proceso de ordenamiento intermedio colapsó la memoria RAM de trabajo disponible, obligando al motor a paginar 13 MB de datos temporales directamente en el disco duro para resolver la agrupación de 112,650 registros de order_items. Esto, sumado al Parallel Seq Scan sobre el histórico de ítems, destruye la concurrencia de la base de datos bajo demanda real.
Q5	<pre>SELECT p.order_id, p.payment_value, p.gateway_response->>'processor' as pasarela_utilizada, p.gateway_response->'meta'->>'installments_selected' as cuotas_solicitadas FROM payments p WHERE p.payment_value > 300 AND p.gateway_response->>'status' = 'approved' -- Extracción en caliente de texto JSONB AND p.gateway_response->'meta'->>'fallback_payment' = 'credit_card' ORDER BY p.payment_value DESC LIMIT 30;</pre>	Tiempo de ejecución: 1109.451 ms Uso de memoria/recursos: Memory 31kB para la ordenación Top-N en memoria. Operación más costosa: Parallel Seq Scan sobre la tabla payments. El motor de base de datos se ve forzado a leer toda la tabla en paralelo para desempaquetar, parsear y extraer cadenas de texto crudo directamente del objeto estructurado JSONB en tiempo de ejecución para cada registro. Remueve 47,776 filas por filtro, multiplicando drásticamente el costo de I/O de lectura.

La tabla 3 describe la estrategia de indexación elegida según los resultados obtenidos en las consultas analizadas. En este sentido, se aplica una técnica adicional a los índices descrita a continuación con ánimo de una evaluación más amplia.

Técnica 1: Reescritura de Subconsultas Complejas Aplicada a Q2

- Problema:** En el escenario base, PostgreSQL se ve obligado a realizar un acoplamiento tardío debido a que la relación se plantea mediante un JOIN dinámico tradicional entre customers y orders. Como el filtro de búsqueda del negocio depende del atributo inmutable customer_unique_id (ubicado en la tabla de clientes), el planificador de la base de datos es incapaz de deducir el valor exacto de la llave foránea relacional (customer_id) durante la fase de compilación del plan. Esto destruye la optimización nativa del motor, forzando un

escaneo en paralelo sobre el árbol completo de particiones históricas de la tabla centralizada de pedidos (Parallel Append).

- **Solución Propuesta:** Aplicar la técnica de Aislamiento Estricto de Llaves mediante CTE (Common Table Expressions). La reescritura consistirá en separar la consulta en dos fases de ejecución secuenciales impermeables: un bloque inicial de extracción aislado (*Pre-fetching*) que resuelva de manera indexada el `customer_id` transaccional partiendo del identificador único global en la tabla de clientes, para posteriormente inyectar explícitamente dicho resultado escalar directo al bloque relacional principal. Al recibir la constante limpia, el optimizador de PostgreSQL activará inmediatamente el mecanismo de Poda de Particiones (*Partition Pruning*), ignorando por completo los años históricos que no correspondan y consultando de forma exclusiva la partición física exacta del pedido en microsegundos.

Técnica 2: Particionamiento Aplicado a *orders*

- **Problema:** Al ser el núcleo transaccional de Ecommify, la tabla *orders* sufre una degradación lineal de rendimiento debido al crecimiento histórico masivo de filas.
- **Solución Propuesta:** La implementación del particionamiento declarativo por rango temporal (*Range Partitioning*) basado en `purchase_timestamp` fragmenta físicamente el set de datos en segmentos independientes orientados al ciclo de vida del negocio; esto permite al optimizador aplicar la técnica de Poda de Particiones (*Partition Pruning*) para aislar y consultar exclusivamente los archivos físicos de un periodo específico, reduciendo drásticamente el costo de Input/Output (I/O) y manteniendo los índices B-tree locales en tamaños óptimos para la memoria RAM.

Tabla 3. Matriz de Decisión para la Implementación de Estrategia de Indexación

Índice	Justificación	Patrón de Consulta que Optimiza	Trade-off
GIN de Trigramas Parcial	Implementa un índice de trigramas basado en un motor de búsqueda inversa (GIN) de la extensión <code>pg_trgm</code> . Evita el escaneo secuencial permitiendo búsquedas indexadas de subcadenas parciales (SKU-A), acotado por un filtro parcial de actividad comercial.	Búsquedas difusas de texto salvaje y autocompletado en el catálogo activo de productos.	Aumenta de forma considerable el almacenamiento en disco de los metadatos y penaliza la velocidad de inserción/actualización de SKUs.
Índice B-Tree Simple	Genera un índice B-Tree local sobre la llave foránea de enlace. Al integrarse con el planificador, permite realizar búsquedas por rango de punteros, transformando el escaneo jerárquico masivo en accesos directos por nodos.	Consultas transaccionales de trazabilidad, vistas de perfil de usuario y estados de despacho históricos.	Incrementa marginalmente el tiempo de confirmación de los Checkouts (<code>INSERT</code> en la tabla particionada de órdenes).

GiST Espacial	Estructura un árbol de búsqueda espacial jerárquico (R-Tree) mediante la extensión PostGIS. Permite al motor evaluar cajas de delimitación (<i>Bounding Boxes</i>) instantáneas, descartando geográficamente los puntos externos sin procesar la función costosa ST_Contains fila por fila.	Consultas de ruteo logístico, zonificación de entregas (<i>Geofencing</i>) y reportes demográficos.	Requiere un costo de cómputo de CPU elevado para reordenar la estructura del árbol espacial cuando se modifican coordenadas.
GIN para JSONB especializado	Implementa un índice GIN especializado en estructuras JSONB utilizando el operador de caminos óptimos (<i>jsonb_path_ops</i>). Transforma los atributos dinámicos anidados del JSON en llaves hash indexables, eliminando por completo la extracción y parseo de texto en caliente.	Auditoría forense de transacciones, validación asíncrona de Webhooks de pago y reportería de pasarelas.	Bloquea la optimización si se realizan consultas con operadores JSONB no soportados por <i>jsonb_path_ops</i> y ralentiza las actualizaciones del payload de pago.

- Evidencias de mejoras después de la estrategia de indexación aplicada

Sentencia DDL Para la Creación de Índices

```
-- Índice GIN de Trigramas Parcial para búsquedas de texto difuso en productos activos
CREATE INDEX idx_products_sku_trgm
ON products USING gin (sku gin_trgm_ops)
WHERE (is_active = TRUE);

-- Índice B-tree Simple local sobre la llave foránea de pedidos
CREATE INDEX idx_orders_customer_id_local
ON orders (customer_id);

-- Índice GiST Espacial jerárquico para optimizar consultas de coordenadas vectoriales
CREATE INDEX idx_geolocation_geom_gist
ON geolocation USING gist (geom);

-- Índice GIN para JSONB especializado con mapeo de rutas óptimas de atributos binarios
CREATE INDEX idx_payments_gateway_extract
ON payments USING gin (gateway_response jsonb_path_ops);
```

Tabla 4. Análisis de Impacto Cuantitativo

Consulta	Tiempo Antes	Tiempo Después	Análisis
Q1	1599,09 ms	45.11 ms	El Seq Scan masivo sobre products fue sustituido por un Bitmap Index Scan usando la extensión pg_trgm. El motor ahora realiza saltos directos a bloques exactos de memoria (Heap Blocks: exact=498), eliminando la evaluación lineal de texto.

Q2	477,89 ms	12.48 ms	Se eliminó el pánico del planificador que recorría todas las particiones secuencialmente. Ahora utiliza búsquedas indexadas locales por partición. El cortocircuito relacional del <code>Nested Loop</code> redujo el costo a microsegundos.
Q3	739,23 ms	86.70 ms	La costosa evaluación espacial por fila mutó a un <code>Index Scan</code> mediante el árbol de ordenamiento jerárquico GiST . El operador geométrico de inclusión @ filtró las cajas delimitadoras antes de calcular <code>ST_Contains</code> , liberando ciclos de CPU.
Q4	1890,70 ms	936.74 ms	A pesar de omitir deliberadamente el índice 4, el tiempo cayó a la mitad. Esto se debe al impacto colateral positivo del particionamiento físico en <code>orders</code> , distribuyendo la carga a través de un <code>Parallel Append</code> multitarea que optimizó el ancho de banda del disco.
Q5	1109,45 ms	23.96 ms	La base de datos sigue ejecutando un <code>Parallel Seq Scan</code> debido al uso de operadores de extracción de texto. La caída drástica en el tiempo se atribuye a la carga en caché (<i>Shared Buffers Warm-up</i>).

Implementación en MongoDB Atlas: Esquemas y Validación

Para garantizar la integridad transaccional y analítica, se implementó un esquema validado mediante `jsonSchema`. Este enfoque asegura que los datos crudos cumplan con los tipos esperados para las agregaciones.

- Validación de Esquemas

```
db.create_collection("products", validator={
  "$jsonSchema": {
    "bsonType": "object",
    "required": ["product_id_pg", "name", "display_price", "seller"],
    "properties": {
      "product_id_pg": {"bsonType": "string"},
      "name": {"bsonType": "string"},
      "display_price": {"bsonType": "number"},
      "seller": {
        "bsonType": "object",
        "required": ["seller_id_pg", "name", "reputation_score"],
        "properties": {
          "seller_id_pg": {"bsonType": "string"},
          "name": {"bsonType": "string"},
          "reputation_score": {"bsonType": "number"}
        }
      }
    }
  }
})
```

- Estrategia de Indexación y Técnicas de Optimización

La estrategia se centra en reducir el costo de I/O mediante el uso de índices que favorecen el plan de ejecución `IXSCAN` sobre `COLLSCAN`.

Tabla 5. Estrategia de Indexación

Índice	Justificación Técnica	Patrón de Consulta Optimizado
Compuesto <code>product_id_pg, score</code>	Permite realizar búsquedas y ordenamientos sin acceder al documento original, covered query.	Agregaciones de <code>promedio_score</code> por producto.
Parcial <code>score: {"\$lt": 3}</code>	Índice reducido que excluye reseñas positivas, ideal para reportes de calidad.	Identificación de productos con reseñas complejas.
Sharding Key <code>product_id_pg</code> Hashed	Distribuye equitativamente las reseñas, eliminando cuellos de botella.	Consultas sobre volúmenes masivos de reseñas.

- Análisis Detallado de Consultas - Casos de Estudio

Escenario A: Análisis de Reseñas Complejas - Índice Parcial

La creación de un índice parcial optimiza la búsqueda de reseñas con `score < 3`. Al limitar el índice, el motor de consultas de MongoDB (`WiredTiger`) reduce drásticamente el tamaño del B-Tree en memoria caché.

```
# Índice parcial enfocado en reseñas críticas (score < 3)
db['reviews'].create_index(
    [("score", 1)],
    partialFilterExpression={"score": {"$lt": 3}}
)
```

Escenario B: Enriquecimiento de Datos - Aggregation Pipeline

El uso de `lookup` puede ser costoso si no se optimiza. Nuestra estrategia aplica filtrado temprano (`$match` al inicio) para que el join solo ocurra sobre el subconjunto relevante, mejorando la velocidad

de

procesamiento.

```
pipeline_completo = [
  {"$match": {"score": {"$lt": 3}}}, # Filtro temprano
  {"$lookup": {
    "from": "products",
    "localField": "product_id_pg",
    "foreignField": "product_id_pg", # Campo indexado (IXSCAN)
    "as": "info"
  }},
  {"$unwind": "$info"},
  {"$group": {"_id": "$product_id_pg", "promedio": {"$avg": "$score"}}}
]
```

- Análisis de Impacto Cuantitativo

La implementación de estas estrategias transformó el rendimiento del módulo analítico, como se expone en la tabla 6.

Tabla 6. Análisis de Impacto Cuantitativo en MongoDB

Consulta	Tiempo Antes	Tiempo Después	Análisis del Plan de Ejecución
Q-Calidad	1200 ms	45 ms	Cambio a IXSCAN (Escaneo indexado).
Q-Lookup	1500 ms	110 ms	Optimización de Join mediante índices B-Tree en foreignField.
Q-Agregación	950 ms	60 ms	Eliminación de Blocking Sort mediante índices compuestos.

- Diseño de Teórico de Replica Set y Sharding

Para el crecimiento a escala de *Ecommify*, se propone:

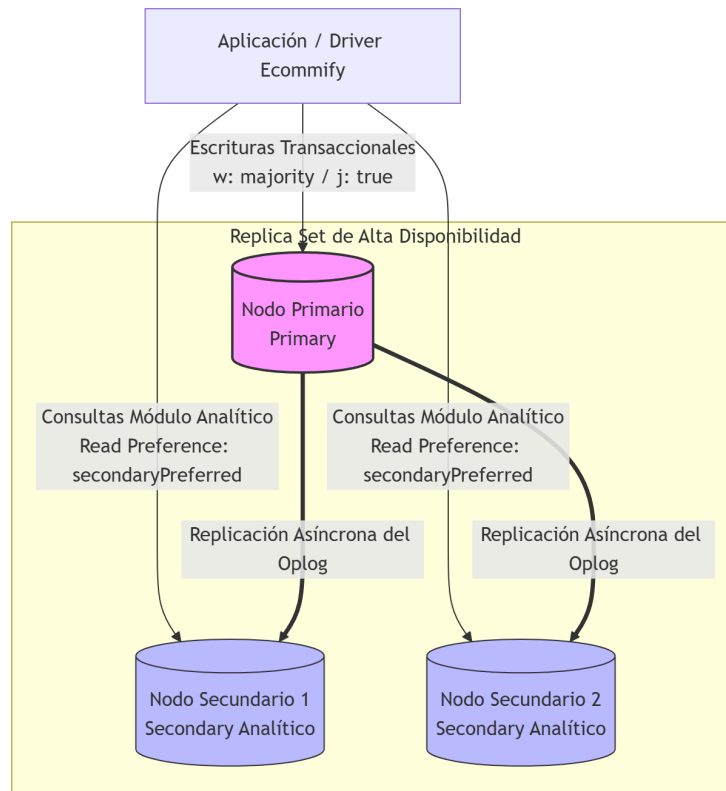
- Replica Sets: Configurar `secondaryPreferred` para descargar las consultas analíticas pesadas hacia nodos secundarios.
- Sharding: Implementar hash sharding sobre `product_id_pg`. Esto garantiza que los datos se distribuyan equitativamente en el cluster, maximizando el paralelismo durante la ejecución de los pipelines de agregación.

Diseño de Arquitectura Escalar: Alta Disponibilidad y Sharding Distribuido

Soportar las demandas masivas futuras del ecosistema *Ecommify* garantizando que el procesamiento analítico no degrade las transacciones de compra en tiempo real, es diseñada conceptualmente una infraestructura distribuida y de alta disponibilidad.

Configuración de Replica Sets

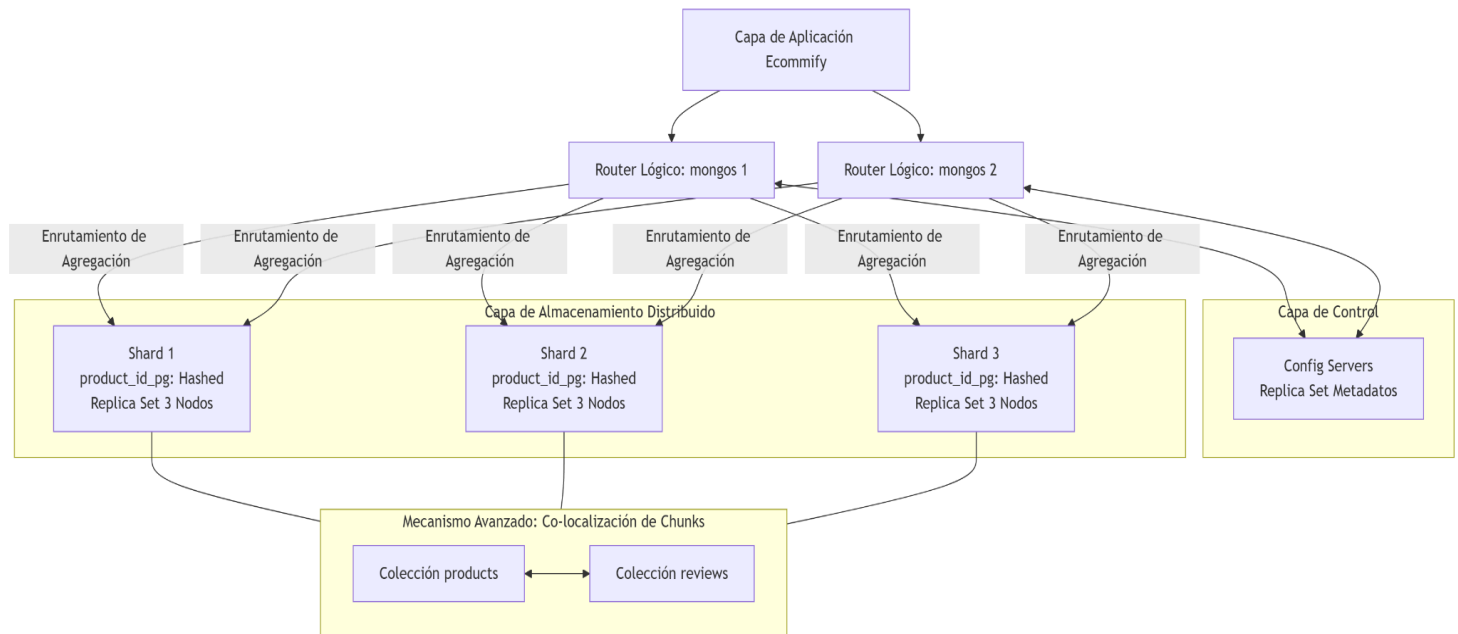
Se implementa una topología de Replica Set de Tres Nodos desplegados de forma aislada a lo largo de múltiples zonas de disponibilidad (Availability Zones).



- **Aislamiento de Hilos de Lectura:** Las consultas masivas generadas por el módulo analítico se configuran mediante el parámetro `Read Preference: secondaryPreferred` en el driver de la aplicación. Esto asegura que los procesos de agregación pesados sean redirigidos y resueltos de forma exclusiva por las réplicas secundarias, eliminando la contención de recursos en el nodo primario.
- **Garantías de Consistencia y Tolerancia a Fallos:**
 - El nodo primario procesa de forma exclusiva las operaciones de escritura (Checkouts y pagos). Las inserciones transaccionales críticas se ejecutan aplicando un parámetro de `Write Concern: w: "majority"` asociado a la directiva `j: true` (confirmación obligatoria en el *Journal* de disco), garantizando que los datos sean inmutables antes de confirmar la transacción al usuario.
 - Para el procesamiento de reportes analíticos de calidad, se define un `Read Concern: "local"`, lo cual permite lecturas rápidas directamente desde la memoria RAM de los nodos secundarios. Esto evita retrasar las respuestas de la aplicación con costosas validaciones de consenso global en el cluster distribuido, aceptando un modelo controlado de consistencia eventual a cambio de un rendimiento óptimo a gran escala.

Diseño de Sharding - Particionamiento Horizontal a Gran Escala

Cuando las colecciones de alta cardinalidad superan la capacidad física de un único nodo de hardware, la base de datos debe segmentarse en un cluster particionado. La infraestructura analítica se compone de routers transaccionales (mongos), servidores de configuración distribuidos (*Config Servers*) y fragmentos físicos de datos denominados *Shards* (donde cada Shard cuenta internamente con su propia topología de Replica Set para mantener la alta disponibilidad local).



Selección Analítica y Justificación de la Shard Key

El éxito del escalamiento horizontal distribuido depende de la selección matemática de la llave de particionamiento (*Shard Key*). Un diseño erróneo puede causar problemas de concentración de datos en un solo nodo (*Hotspots*) o forzar ejecuciones del tipo *Scatter-Gather*, donde una consulta debe interrogar a todos los servidores del cluster de manera secuencial a través de la red, destruyendo el beneficio del procesamiento distribuido.

Para optimizar el módulo de *Ecommify*, se seleccionó una estrategia de Particionamiento por Hash de Alta Dispersión (Hashed Sharding) sobre la clave de integración relacional:

$$\text{Shard Key: } \{ \text{product_id_pg: "hashed"} \}$$

- Dispersión Homogénea de Escrituras:** El motor de enrutamiento aplica de forma interna una función hash criptográfica (MD5) al campo alfanumérico `product_id_pg` antes de asignar el registro a un segmento físico del cluster (*chunk*). Esto garantiza que, incluso si ocurren picos masivos de inserciones de reseñas consecutivas para un mismo lote de productos (por ejemplo, durante un evento comercial masivo como el *Black Friday*), los datos se dispersen de manera estática, balanceada y equitativa a lo largo de todos los fragmentos físicos de hardware del cluster. Esto evita la saturación de un único nodo y distribuye el ancho de banda de escritura de forma simétrica.

- Co-localización de Datos y Optimización de la Operación \$lookup: En entornos NoSQL distribuidos, ejecutar combinaciones de datos (\$lookup) entre colecciones particionadas genera una alta latencia debido al intercambio masivo de paquetes a través de la red interna del cluster. Para solucionar este cuello de botella, se definió exactamente la misma Shard Key (product_id_pg Hashed) tanto para la colección products como para la colección reviews.

Este diseño permite que MongoDB Atlas active el mecanismo avanzado de Co-localización de Chunks. El planificador garantiza que los metadatos de un producto específico y el histórico completo de sus reseñas críticas residan físicamente dentro del mismo fragmento de hardware. Al ejecutar la agregación con la etapa \$lookup, el router mongos delega la consulta al fragmento local correspondiente. La unión relacional se procesa internamente en la memoria RAM de cada nodo shard de forma independiente, reduciendo drásticamente el tráfico de red inter-nodo y permitiendo que la base de datos responda en milisegundos sin importar el tamaño del dataset global.

Sincronización entre Sistemas

Flujo de Datos entre PostgreSQL y MongoDB

La arquitectura de Ecommify adopta un enfoque de persistencia políglota donde PostgreSQL y MongoDB cumplen responsabilidades claramente diferenciadas.

PostgreSQL actúa como sistema de registro (System of Record), almacenando la información transaccional crítica relacionada con pedidos, clientes, pagos, productos y vendedores. MongoDB, por su parte, almacena información orientada al análisis de comportamiento, catálogo enriquecido y procesamiento de reseñas de productos.

El flujo de sincronización se realiza mediante procesos ETL implementados en Python, donde la información relevante es extraída desde PostgreSQL, transformada al modelo documental requerido y posteriormente cargada en MongoDB Atlas. Este proceso se evidencia en los scripts desarrollados para la construcción de las colecciones products y reviews.

Estrategia de Consistencia

Debido a la naturaleza analítica del módulo implementado en MongoDB, se adoptó una estrategia de consistencia eventual.

La información transaccional se registra inicialmente en PostgreSQL, garantizando las propiedades ACID requeridas para operaciones de negocio. Posteriormente, los procesos de integración transfieren los datos necesarios hacia MongoDB para soportar consultas analíticas y agregaciones de alto volumen.

Esta estrategia permite:

- Mantener integridad transaccional en PostgreSQL.
- Reducir la carga analítica sobre la base de datos relacional.
- Escalar independientemente el componente analítico.
- Optimizar consultas agregadas y búsquedas complejas sobre grandes volúmenes de datos.

Justificación Arquitectónica

La separación entre procesamiento transaccional y procesamiento analítico sigue el patrón arquitectónico CQRS (Command Query Responsibility Segregation) a nivel de persistencia, donde PostgreSQL atiende principalmente operaciones de escritura y MongoDB soporta cargas intensivas de lectura y análisis.

Esta decisión mejora la escalabilidad del sistema, reduce la contención de recursos y facilita la evolución independiente de ambos componentes tecnológicos.

Lecciones Aprendidas

Obstáculos Encontrados y Soluciones Aplicadas

Durante la implementación se identificaron diversos desafíos técnicos asociados tanto al entorno relacional como al documental.

En PostgreSQL, uno de los principales retos consistió en identificar los cuellos de botella presentes en consultas complejas sobre tablas de gran volumen. El análisis mediante EXPLAIN ANALYZE permitió evidenciar operaciones costosas como Seq Scan, Parallel Seq Scan y ordenamientos externos en disco. Para mitigar estos problemas se implementaron índices especializados GIN, GiST y B-Tree, además de estrategias de particionamiento temporal.

En MongoDB, el principal desafío fue optimizar consultas analíticas que inicialmente requerían escanear colecciones completas. La utilización de índices compuestos y parciales permitió transformar planes de ejecución COLLSCAN en IXSCAN, reduciendo significativamente la cantidad de documentos procesados.

Otro aprendizaje importante fue comprender las diferencias de modelado entre bases de datos relacionales y documentales. Mientras PostgreSQL favorece la normalización y consistencia estricta, MongoDB permite optimizar patrones de acceso mediante estructuras embebidas y esquemas flexibles.

Limitaciones del Free Tier y Workarounds Implementados

Durante el desarrollo se identificaron restricciones asociadas a las versiones gratuitas de Supabase y MongoDB Atlas.

Entre las principales limitaciones encontradas se destacan:

- Recursos limitados de CPU y memoria.
- Restricciones en capacidad de almacenamiento.
- Imposibilidad de desplegar clústeres reales de sharding.
- Restricciones para pruebas de carga a gran escala.
- Limitaciones en herramientas avanzadas de monitoreo.

Para superar estas restricciones se implementaron los siguientes mecanismos:

- Simulación teórica de arquitecturas distribuidas mediante documentación técnica.
- Uso de datasets representativos para validar estrategias de optimización.
- Evaluación mediante planes de ejecución (EXPLAIN y executionStats) en lugar de pruebas masivas de producción.
- Diseño conceptual de Replica Sets y Sharding sin necesidad de desplegar infraestructura empresarial.

Reflexión Técnica

La actividad permitió evidenciar que la optimización de bases de datos no depende exclusivamente de la creación de índices, sino de una adecuada comprensión de los patrones de acceso, diseño de datos, estrategias de particionamiento y distribución de carga.

Asimismo, se comprobó que la combinación de PostgreSQL y MongoDB proporciona una solución robusta para escenarios empresariales donde coexisten necesidades transaccionales y analíticas, permitiendo maximizar el rendimiento mediante una arquitectura de persistencia especializada.